Google

pt_regs

the good, the bad and the ugly

LPC 2023

revest@google.com

Nov 2023

01
History



0.01 (1991)

In the beginning of time...

```
_system_call:
        cmpl $nr_system_calls-1,%eax
        ja bad_sys_call
        push %ds
        push %es
        push %fs
        pushl %edx
        pushl %ecx              # push %ebx,%ecx,%edx as parameters
        pushl %ebx              # to the system call
        movl $0x10,%edx         # set up ds,es to kernel space
        mov %dx,%ds
        mov %dx,%es
        movl $0x17,%edx         # fs points to local data space
        mov %dx,%fs
        call _sys_call_table(,%eax,4)
```

Exceptions

Google

0.95 (1992)

PTrace

```
* Stack layout in 'ret_from_system_call':
*      ptrace needs to have all regs on the stack.
*      if the order here is changed, it needs to be
*      updated in fork.c:copy_process, signal.c:do_signal,
*      ptrace.c ptrace.h
*
*       0(%esp) - %ebx
*       4(%esp) - %ecx
*       8(%esp) - %edx
*       C(%esp) - %esi
*      10(%esp) - %edi
*      14(%esp) - %ebp
*      18(%esp) - %eax
*      1C(%esp) - %ds
*      20(%esp) - %es
*      24(%esp) - %fs
*      28(%esp) - %gs
*      2C(%esp) - orig_eax
*      30(%esp) - %eip
*      34(%esp) - %cs
*      38(%esp) - %eflags
*      3C(%esp) - %oldesp
*      40(%esp) - %oldss
```

```
/* this struct defines the way the registers are stored on the
   stack during a system call. */

struct pt_regs {
  long ebx;
  long ecx;
  long edx;
  long esi;
  long edi;
  long ebp;
  long eax;
  long ds;
  long es;
  long fs;
  long gs;
  long orig_eax;
  long eip;
  long cs;
  long eflags;
  long esp;
  long ss;
};
```

UAPI

PTrace

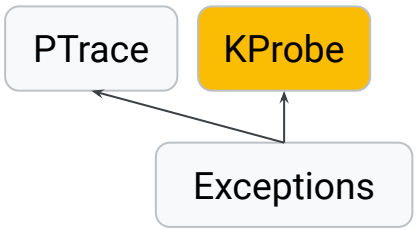
Exceptions

Google

2.6.9 (2004)

KProbe

```
/*
 * Wrapper routine to for handling exceptions.
 */
int kprobe_exceptions_notify(struct notifier_block *self, unsigned long val,
                             void *data)
{
        struct die_args *args = (struct die_args *)data;
        switch (val) {
        case DIE_INT3:
                if (kprobe_handler(args->regs))
                        return NOTIFY_STOP;
                break;
```



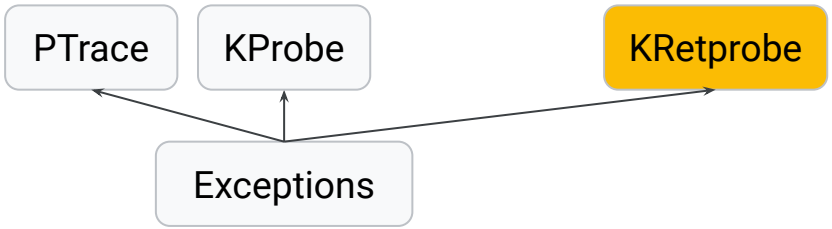
Google

2.6.13 (2005)

KRetprobe

```
/*
 * For function-return probes, init_kprobes() establishes a probepoint
 * here. When a retprobed function returns, this probe is hit and
 * trampoline_probe_handler() runs, calling the kretprobe's handler.
 */
void kretprobe_trampoline_holder(void)
{
        asm volatile (  ".global kretprobe_trampoline\n"
                        "kretprobe_trampoline: \n"
                        "nop\n");

}
```



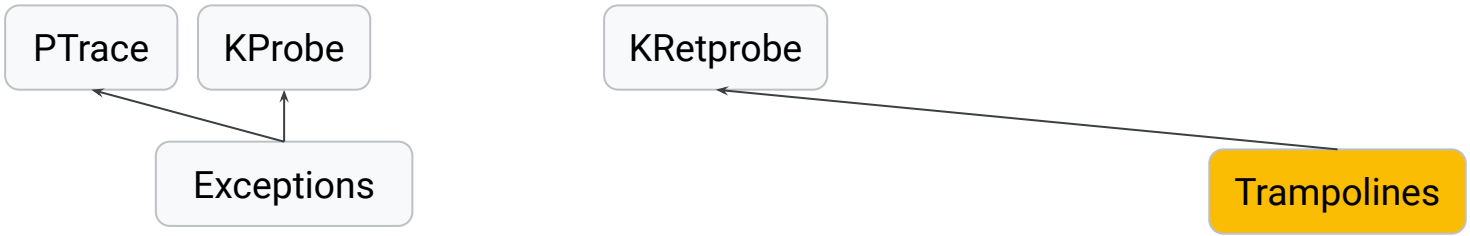
Google

2.6.17 (2006)

KRetprobe-booster

```
/*
 * For function-return probes, init_kprobes() establishes a probepoint
 * here. When a retprobed function returns, this probe is hit and
 * trampoline_probe_handler() runs, calling the kretprobe's handler.
 */
void __kprobes kretprobe_trampoline_holder(void)
{
        asm volatile ( ".global kretprobe_trampoline\n"
                       "kretprobe_trampoline: \n"
                       "       pushf\n"
                       /* skip cs, eip, orig_eax, es, ds */
                       "       subl $20, %esp\n"
                       "       pushl %eax\n"
                       "       pushl %ebp\n"
                       "       pushl %edi\n"
                       "       pushl %esi\n"
                       "       pushl %edx\n"
                       "       pushl %ecx\n"
                       "       pushl %ebx\n"
                       "       movl %esp, %eax\n"
                       "       call trampoline_handler\n"
```

sparse pt_regs



Google

2.6.27 (2008), SAVE_REGS 3.6 (2012), ftrace_regs 5.11 (2021)

FTrace

```
ENTRY(mcount)
        pushl %eax
        pushl %ecx
        pushl %edx
        movl 0xc(%esp), %eax
        subl $MCOUNT_INSN_SIZE, %eax

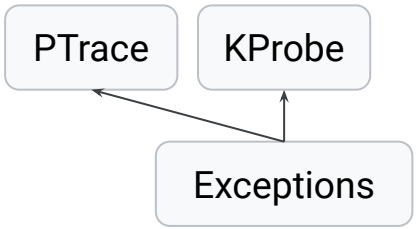
.globl mcount_call
mcount_call:
        call ftrace_stub

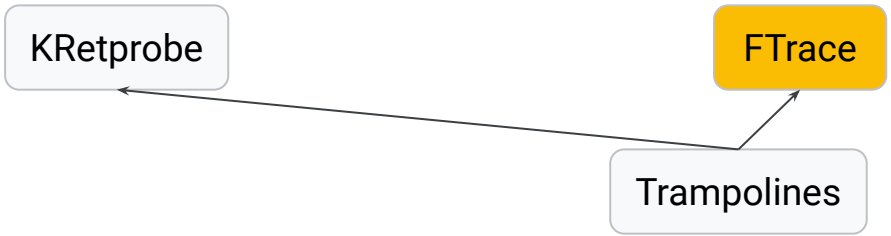
        popl %edx
        popl %ecx
        popl %eax

        ret
END(mcount)
```

```
#ifdef CONFIG_HAVE_DYNAMIC_FTRACE_WITH_ARGS
struct ftrace_regs {
        struct pt_regs          regs;
};

static __always_inline struct pt_regs *
arch_ftrace_get_regs(struct ftrace_regs *fregs)
{
        /* Only when FL_SAVE_REGS is set, cs will be non zero */
        if (!fregs->regs.cs)
                return NULL;
        return &fregs->regs;
}
```



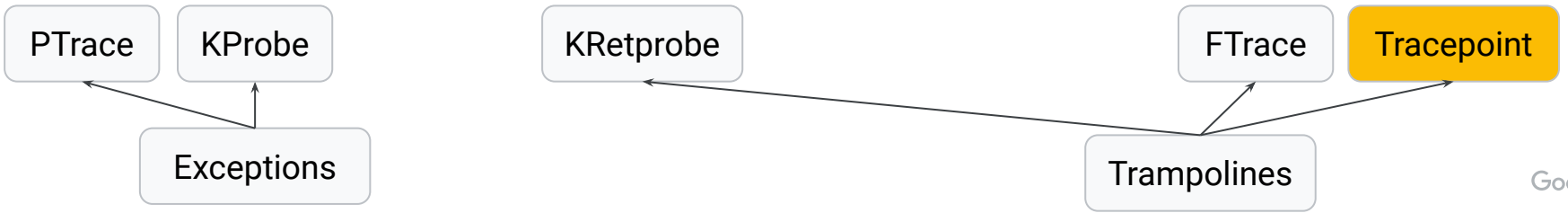


Google

2.6.28 (2008)

Tracepoints

```
#define __DO_TRACE(tp, proto, args)                                     \
        do {                                                            \
                void **it_func;                                         \
                                                                        \
                rcu_read_lock_sched();                                  \
                it_func = rcu_dereference((tp)->funcs);                 \
                if (it_func) {                                          \
                        do {                                            \
                                ((void(*)(proto))(*it_func))(args);     \
                        } while (*(++it_func));                         \
                }                                                       \
                rcu_read_unlock_sched();                                \
        } while (0)
```



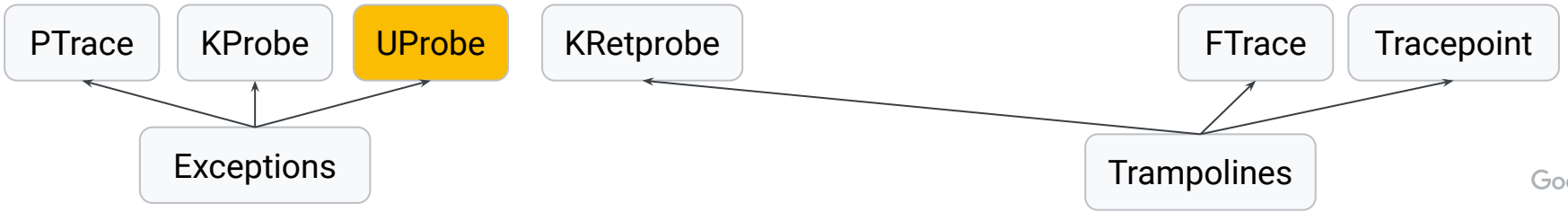
Google

3.5 (2012)

U(ret)Probe

```
/* callback routine for handling exceptions. */
int arch_uprobe_exception_notify(struct notifier_block *self, unsigned long val, void *data)
{
        struct die_args *args = data;
        struct pt_regs *regs = args->regs;
        int ret = NOTIFY_DONE;

        /* We are only interested in userspace traps */
```

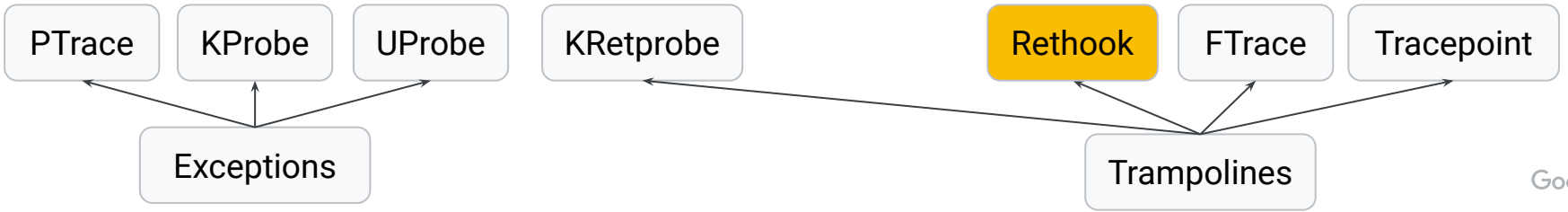


Google

5.18 (2022)

Rethook

```
/*
 * When a target function returns, this code saves registers and calls
 * arch_rethook_trampoline_callback(), which calls the rethook handler.
 */
asm(
        ".text\n"
        ".global arch_rethook_trampoline\n"
        ".type arch_rethook_trampoline, @function\n"
        "arch_rethook_trampoline:\n"
#ifdef CONFIG_X86_64
        ANNOTATE_NOENDBR        /* This is only jumped from ret instruction */
        /* Push a fake return address to tell the unwinder it's a rethook. */
        "       pushq $arch_rethook_trampoline\n"
        UNWIND_HINT_FUNC
        "       pushq $" __stringify(__KERNEL_DS) "\n"
        /* Save the 'sp - 16', this will be fixed later. */
        "       pushq %rsp\n"
        "       pushfq\n"
        SAVE_REGS_STRING
        "       movq %rsp, %rdi\n"
        "       call arch_rethook_trampoline_callback\n"
```

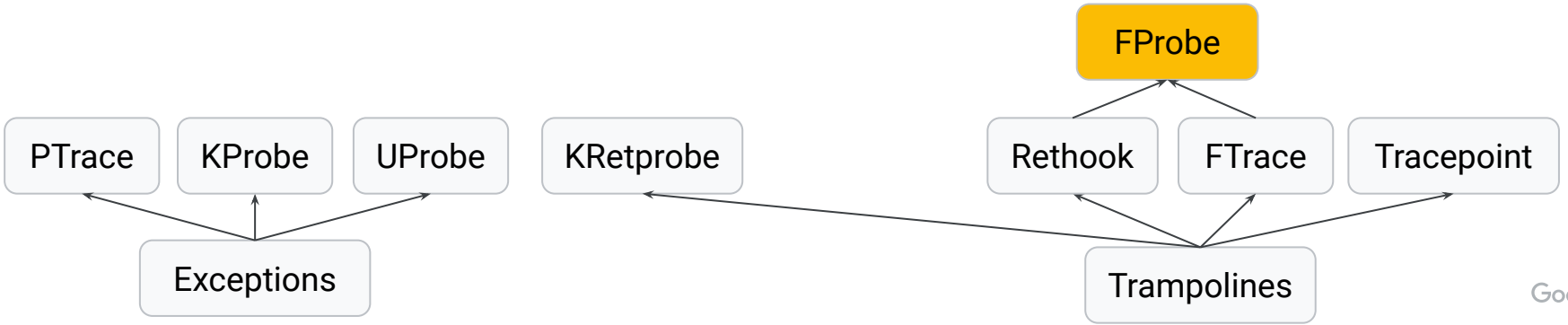


Google

5.18 (2022)

FProbe

```
static void fprobe_init(struct fprobe *fp)
{
        fp->nmissed = 0;
        if (fprobe_shared_with_kprobes(fp))
                fp->ops.func = fprobe_kprobe_handler;
        else
                fp->ops.func = fprobe_handler;
        fp->ops.flags |= FTRACE_OPS_FL_SAVE_REGS;
}
```

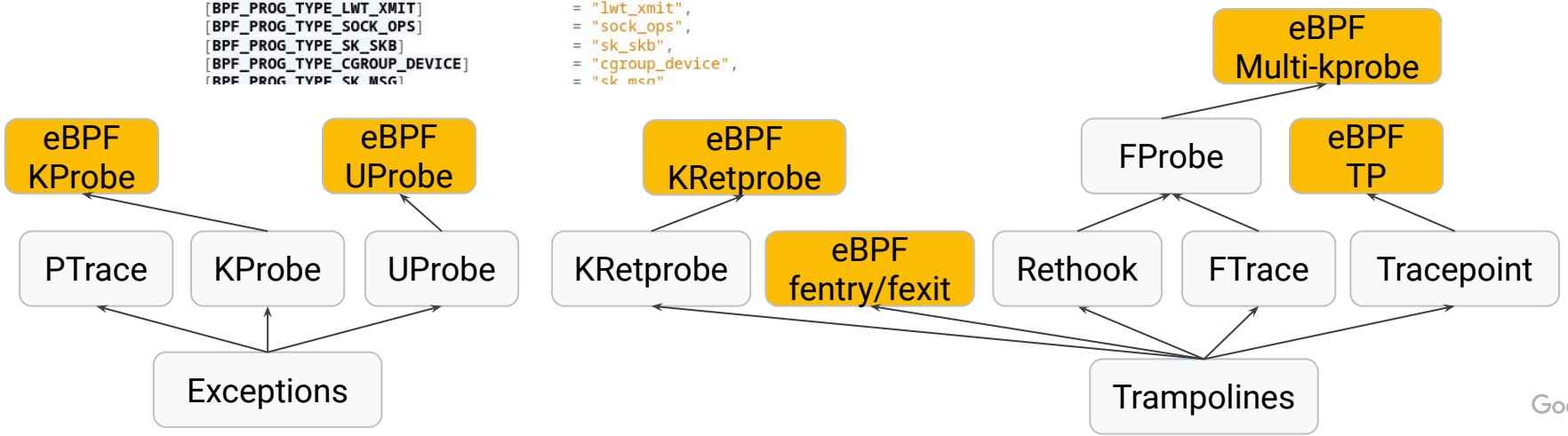


Google

4.0-now (2015, 2016, ...)

eBPF

```
static const char * const prog_type_name[] = {
        [BPF_PROG_TYPE_UNSPEC]                  = "unspec",
        [BPF_PROG_TYPE_SOCKET_FILTER]           = "socket_filter",
        [BPF_PROG_TYPE_KPROBE]                  = "kprobe",
        [BPF_PROG_TYPE_SCHED_CLS]               = "sched_cls",
        [BPF_PROG_TYPE_SCHED_ACT]               = "sched_act",
        [BPF_PROG_TYPE_TRACEPOINT]              = "tracepoint",
        [BPF_PROG_TYPE_XDP]                     = "xdp",
        [BPF_PROG_TYPE_PERF_EVENT]              = "perf_event",
        [BPF_PROG_TYPE_CGROUP_SKB]              = "cgroup_skb",
        [BPF_PROG_TYPE_CGROUP_SOCK]             = "cgroup_sock",
        [BPF_PROG_TYPE_LWT_IN]                  = "lwt_in",
        [BPF_PROG_TYPE_LWT_OUT]                 = "lwt_out",
        [BPF_PROG_TYPE_LWT_XMIT]                = "lwt_xmit",
        [BPF_PROG_TYPE_SOCK_OPS]                = "sock_ops",
        [BPF_PROG_TYPE_SK_SKB]                  = "sk_skb",
        [BPF_PROG_TYPE_CGROUP_DEVICE]           = "cgroup_device",
        [BPF_PROG_TYPE_SK_MSG]                  = "sk_msg",
```



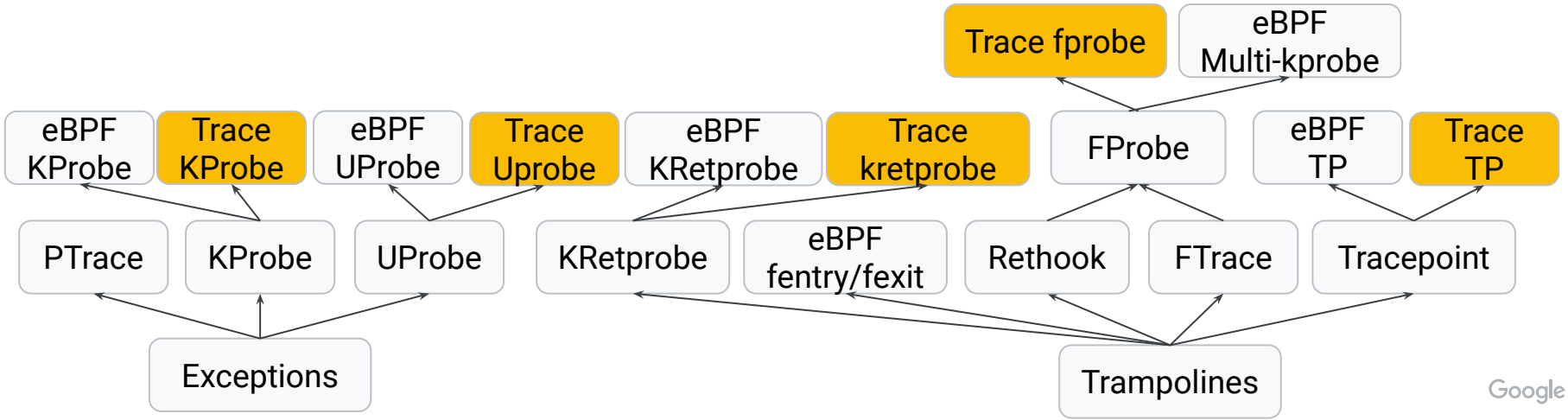
Google

2.6.31-now (2009, ...)

tracefs

```
Usage examples
--------------
To add a probe as a new event, write a new definition to kprobe_events
as below::

  echo 'p:myprobe do_sys_open dfd=%ax filename=%dx flags=%cx mode=+4($stack)' > /sys/kernel/tracing/kprobe_events
```

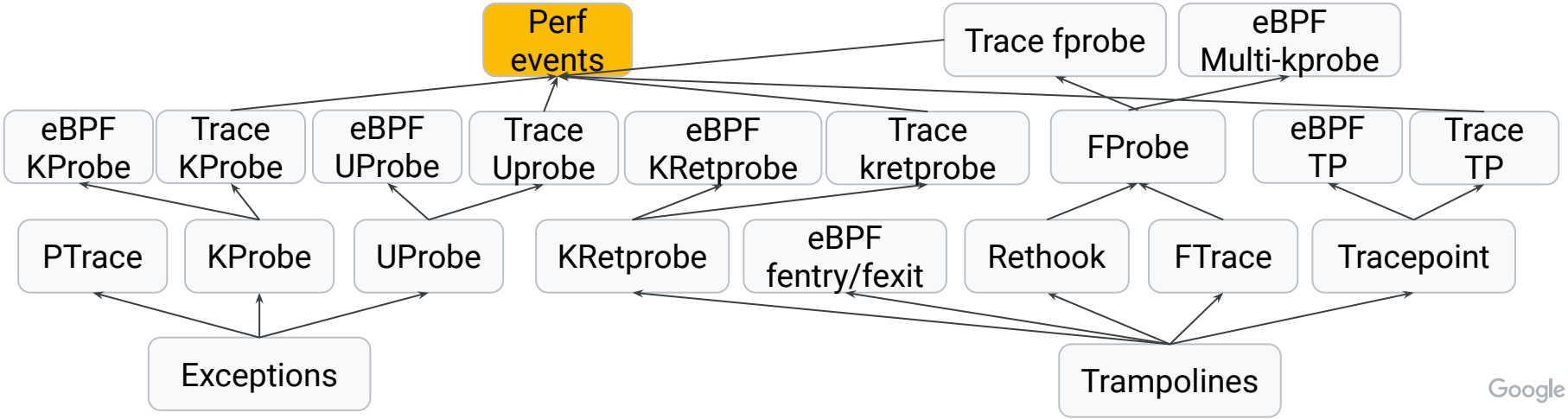


Google

2.6.32-now (2009, ...)

Perf

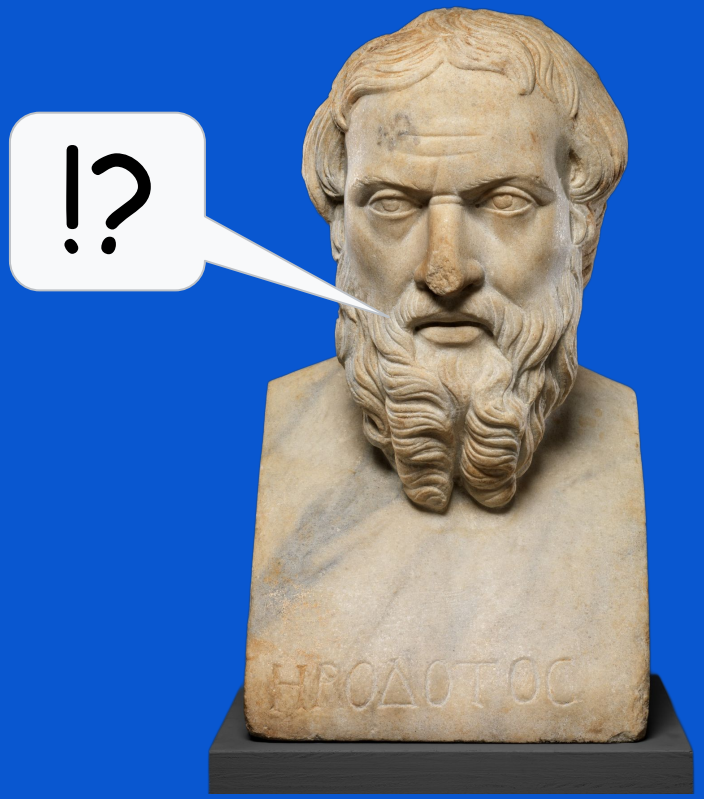
```
/*
 * We abuse bit 3 from flags to pass exact information, see perf_misc_flags
 * and the comment with PERF_EFLAGS_EXACT.
 */
#define perf_arch_fetch_caller_regs(regs, __ip)         {       \
        (regs)->ip = (__ip);                                    \
        (regs)->sp = (unsigned long)__builtin_frame_address(0); \
        (regs)->cs = __KERNEL_CS;                               \
        regs->flags = 0;                                        \
}
```



Google

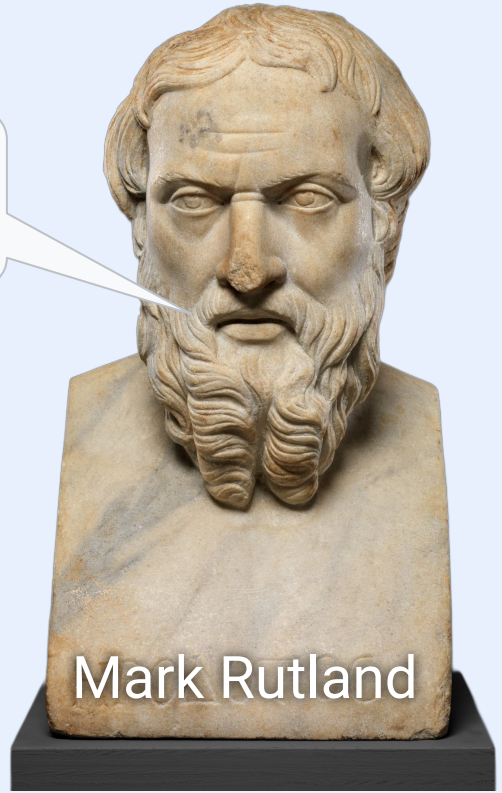
02

Problems



arm64 pt_regs from trampolines

we (incompletely) fill [pt_regs] with bogus values and cannot acquire some state reliably (e.g. PSTATE)



Mapping pt_regs

Full pt_regs

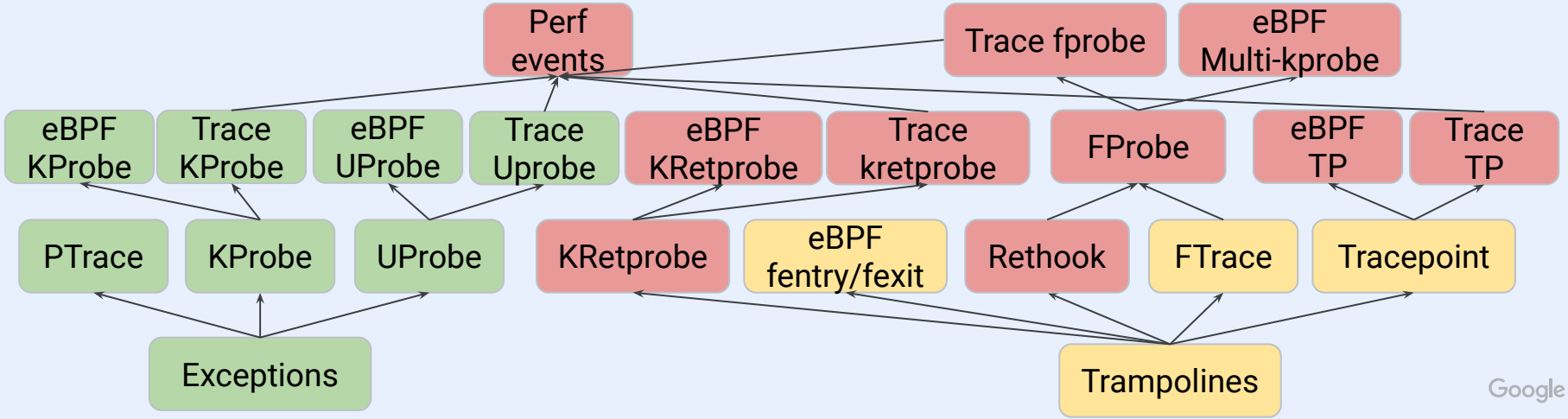
Comes from an exception entry

No pt_regs

Other struct or no struct

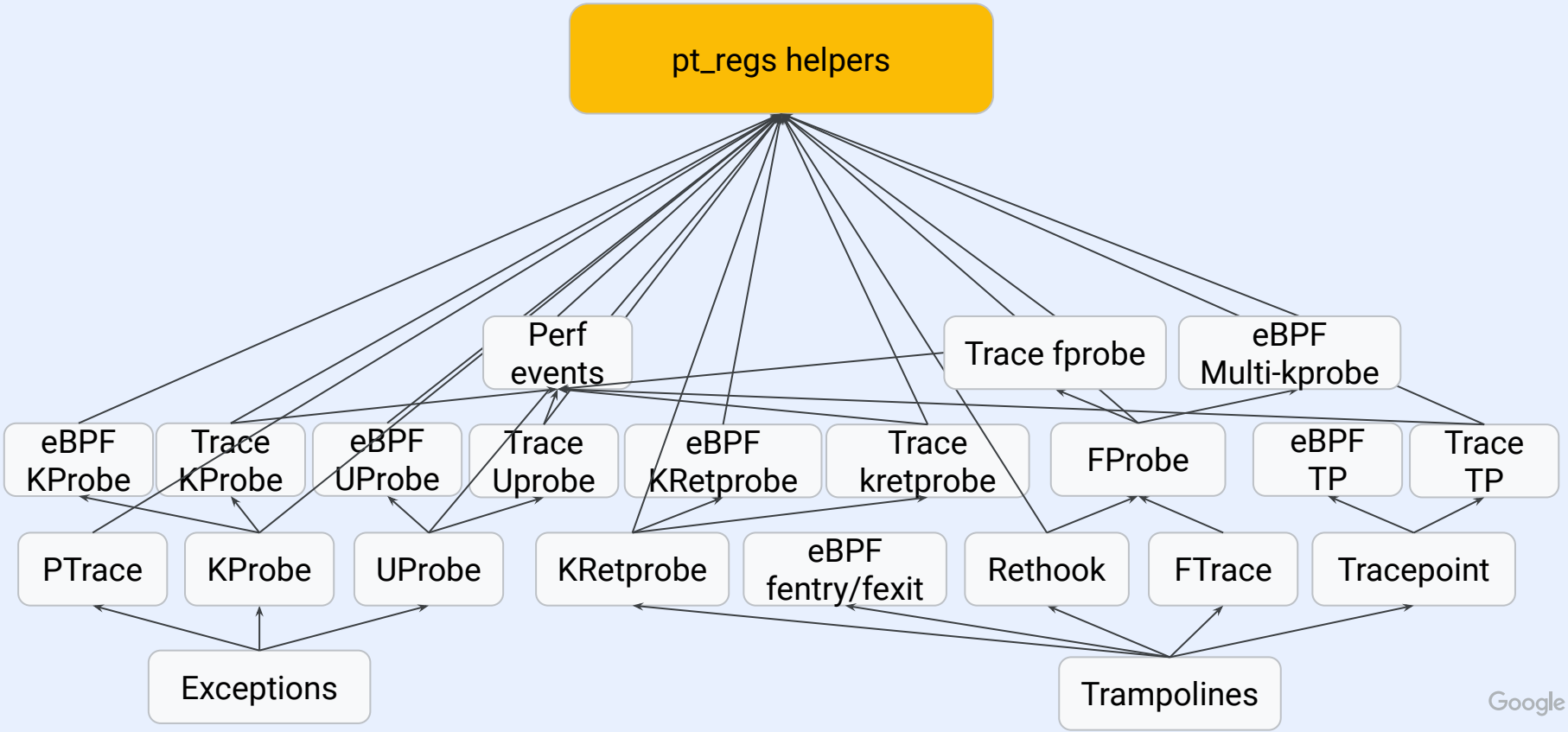
Invalid pt_regs

Sparse or “made up” regs



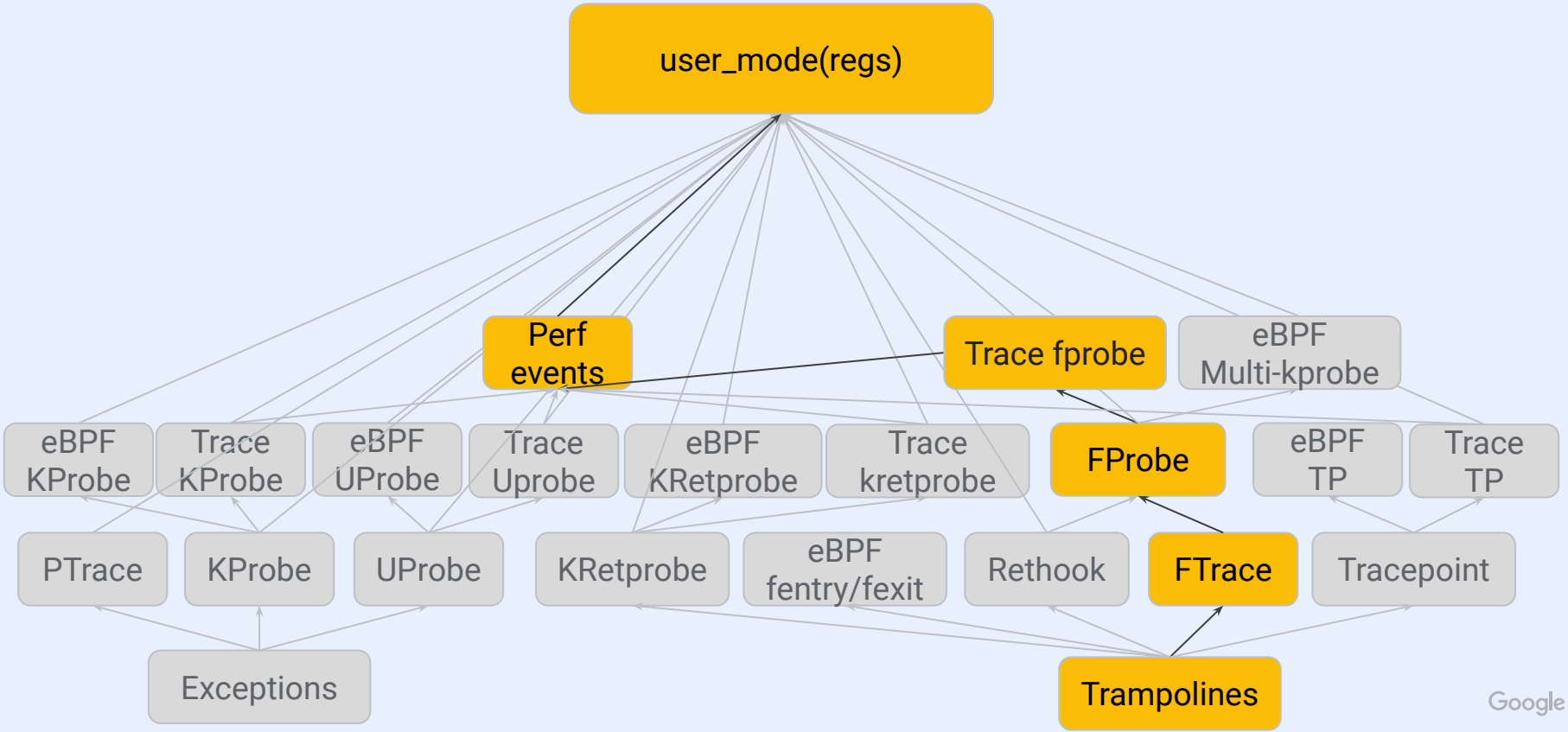
Google

Shared code



Google

Ex possible issue on arm64



Google

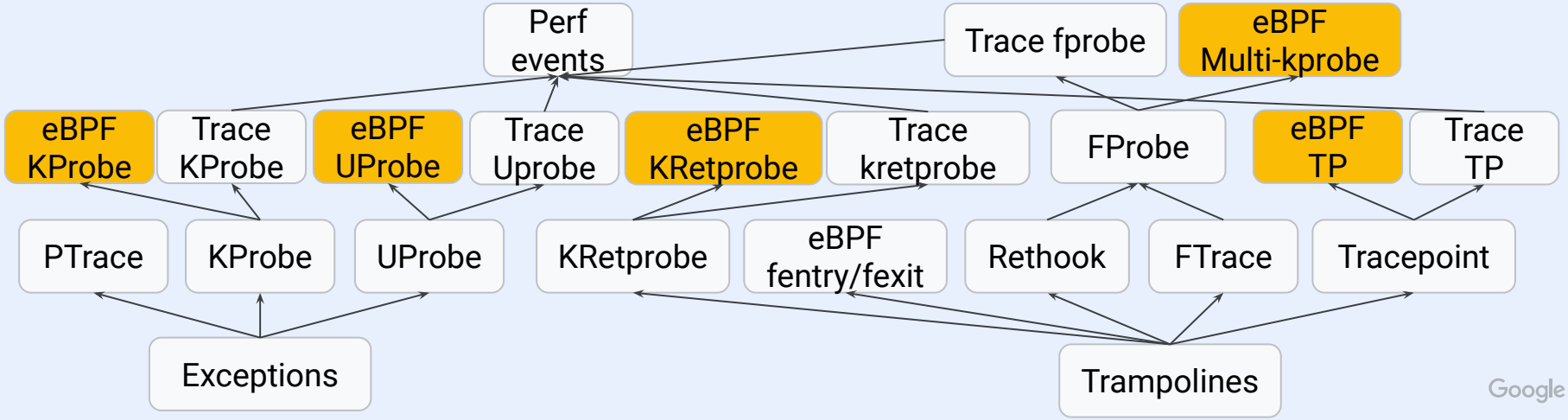
Evolving APIs ?

eBPF programs context

Preferably stays a pt_regs to avoid programs rewrite

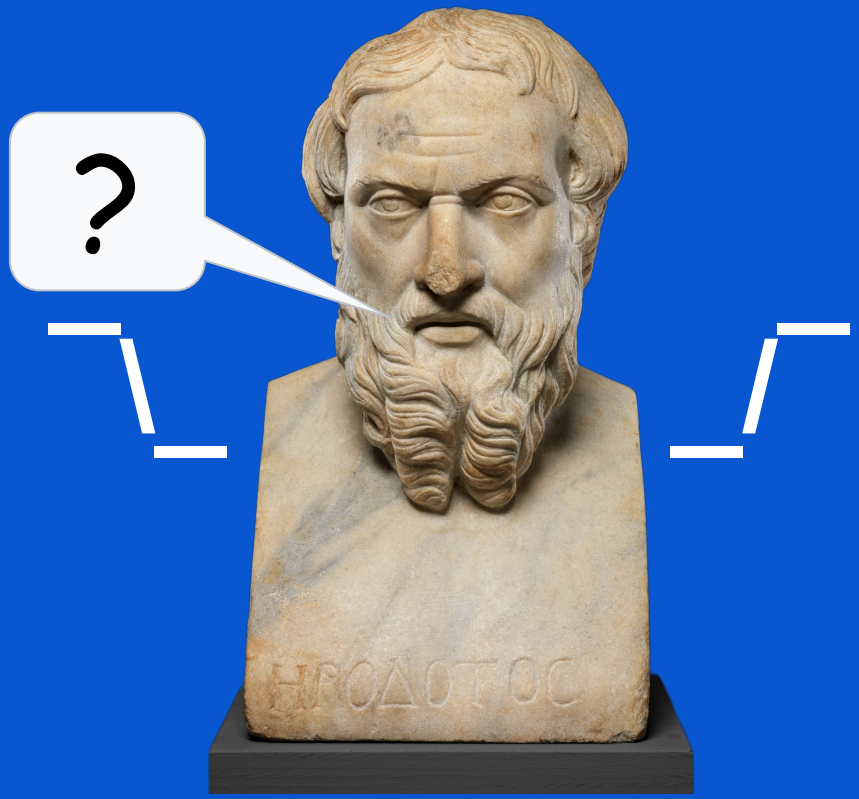
Conversions

Potentially expensive stack copies



Google

03
So what ?



Recommendations

- Never save pt_regs in trampolines
- Avoid using pt_regs when possible
- Keep good documentation
- Be careful